

Name: Bryon Wong Yi Shi

Student number: 230576043

Table of Contents

Part 1: Random Walk Metropolis	2
For part a)	2
For part (b)	3
Part 2: Flight data analysis.....	4
For part a)	4
For part b)	7
For part c)	8

Part 1: Random Walk Metropolis

In this report, we will explain how Metropolis-Hastings algorithm works by implementing the probability density function:

$$f(x) = \frac{1}{2} e^{-|x|}$$

For Rmarkdown, we will use: `f <- function(x) { return(0.5 * exp(-abs(x))) }`

For Python, we will use : `def f(x):
 return 0.5 * np.exp(-np.abs(x))`

For part a)

The algorithm intends to generate $N = 10,000$ samples using the above Metropolis Hastings method with a standard deviation, $s=1$. Next we choose an initial value X_0 and set to 0.

For x_{prev} , we propose a new candidate x_{star} from $N(x_{i-1}, s)$. This computes the acceptance ratio :

$$r = \frac{f(x_{star})}{f(x_{i-1})}$$

Instead of computing the above acceptance ratio, we apply logarithm transformation to avoid numerical problems :

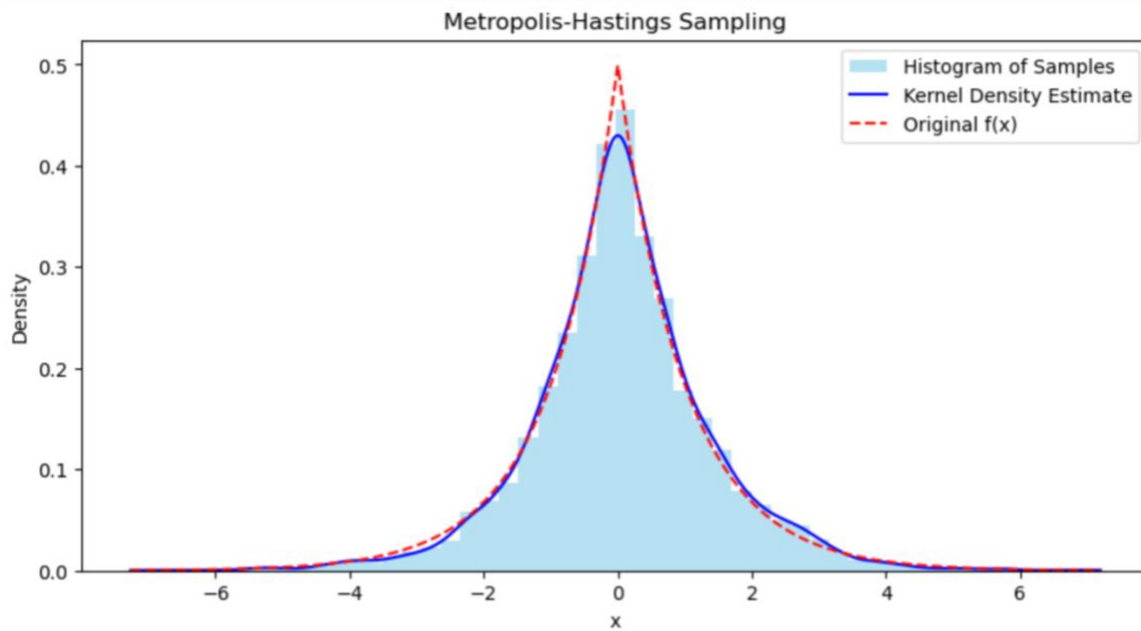
$$\text{Log } r = \log f(x_{star}) - \log f(x_{i-1})$$

Next we generate a uniform random variable u and compare $\log(u)$ with $\log(r)$

If $\log(u) < \log(r)$, accept the proposed x^* , otherwise retain previous value.

Next we use `mean_estimate` and `std_estimate` to provide approximations of the true mean and standard deviation of target distribution. Finally we plot a histogram to show the sampled distribute and a kernel density estimate is added for better visualization.

The original density function is also imposed with the estimated distributions to provide a good comparison.



Monte Carlo estimate of the sample mean: 0.05537750214999279
Monte Carlo estimate of the standard deviation: 1.3486672536456683

For Python:

Monte Carlo estimate of the sample mean, standard deviation: (0.0554, 1.35)

For Rmarkdown:

Monte Carlo estimate of the sample mean, standard deviation: (-0.0944, 1.48)

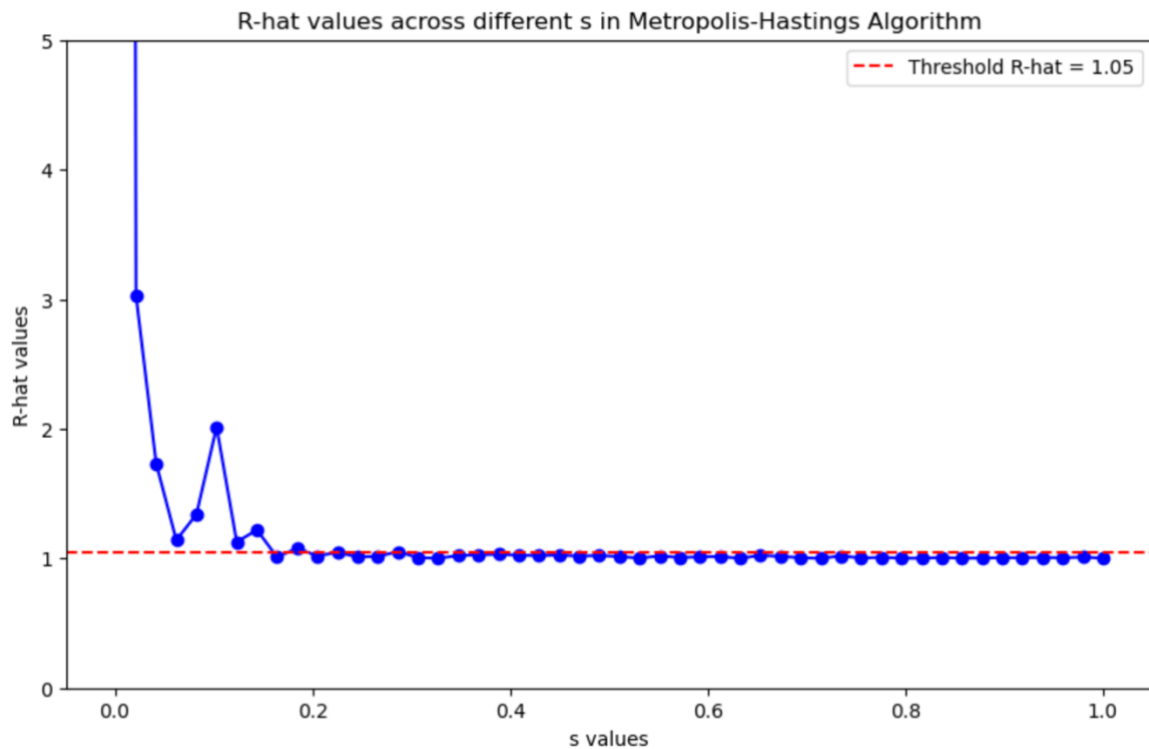
The Monte Carlo estimates (0.0554 in Python, -0.0944 in R) are close to 0 suggesting that both implementations are correctly approximating the expected value.

As KDE(Kernel density estimate) is wider and shorter than original $f(x)$, it could be because KDE smooths out fluctuations from the sample. The bell shaped KDE and histogram aligns with the expected distribution.

For part (b)

We will be using the same algorithm but also implement the convergence diagnostic R-hat to check if multiple independent Markov chains will converge to the same distribution. We generate J independent chains using the `metropolis_hastings` function and store all chains in a 2D array. We define the within and between chain statistics: sample mean of chain, sample variance of chain, overall within-chain variance, overall sample mean, in between-chain sample variance. We also compute R-hat using the `x_star` loop and the logarithm transformation mentioned in part (a) as part of a chain. Using parameters given for the plotting of R-hat values over the grid of s values from 0.001 to 1. With the parameters given as $N=2000$, $J=4$, and $s=0.001$ to 1. Both Python and Rmarkdown have the same steps previously explained. We plot

the graph for how R-hat changes with step size s .



The red dashed line is the threshold $R\text{-hat} = 1.05$.

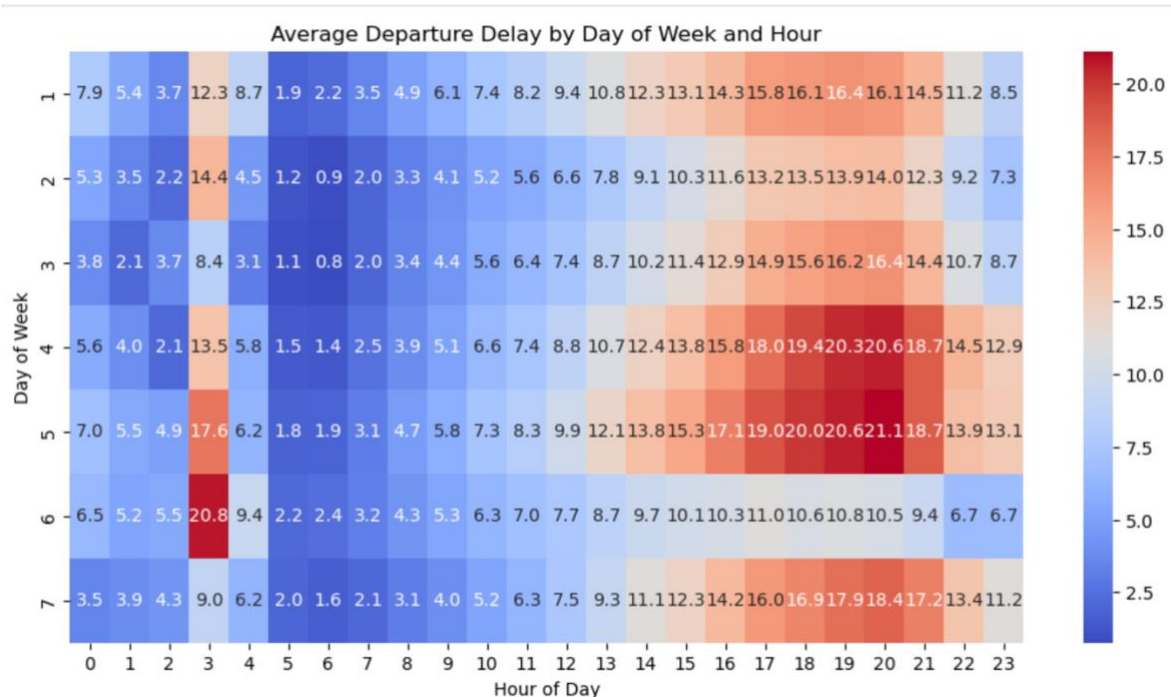
As s increases, $R\text{-hat}$ steadily decreases and converges at 1.05, indicating that the chains start to mix better and approach the target distribution. Beyond $R\text{-hat}=1.05$ does not further improve convergence, thus step size is now optimal for the algorithm.

Part 2:

I will be using flight data from years 2004-2008 to address the 3 parts of the question. We will examine optimal travel times, aircraft age effects and flight diversion predictors.

For part a) : What are the best times and days of the week to minimize delays each year?

For python, I combine flight data into a single dataframe using `pd.concat()`, and drop rows with missing values in key columns (`DepTime`, `ArrTime`, `DepDelay`). Next I extract hour from scheduled departure time (`CSRDepTime`) and create a heatmap showing average departure delays by day of week and hour.



We conclude that delays are highest around the late afternoon to evening from 1500-2200, and Thursday and Fridays have the highest delay.

For Rmarkdown, I begin by loading flight data into SQLite database. I want to identify the patterns by examining scheduled departure times and days of the week. I decided to create barplots for every year based on the best time and best day to fly. I start by creating time bins for 2-hour intervals from scheduled departure times (CSRDeptime) using cut(). Next, I convert numerical day of week values to named factors (Monday-Sunday). Afterwards, I will sum the arrival and departure delays while filtering out non-delayed flights. The best day analysis code functions by looping through each year and filtering the dataset to only include flights from the current year. It counts delayed flights per day of the week and finds the best day through identifying which day has the minimum number of delays. The best time analysis has the same looping structure, but groups by predefined 2-hour time bins (e.g. 0600-0800) and finds the best time by identifying the time range with fewest delays.

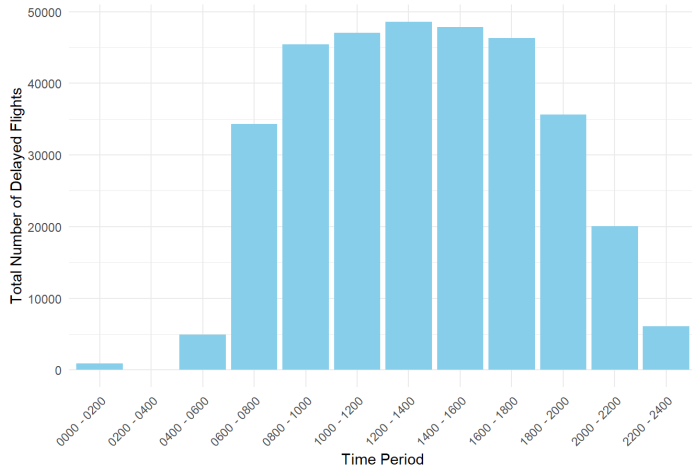
The analysis concluded that Saturday is the best day to fly for 2004-2008, and the best time to fly will be 0200-0400.

```
## Best day to fly in 2004 : Saturday
## Best day to fly in 2005 : Saturday
## Best day to fly in 2006 : Saturday
## Best day to fly in 2007 : Saturday
## Best day to fly in 2008 : Saturday
```

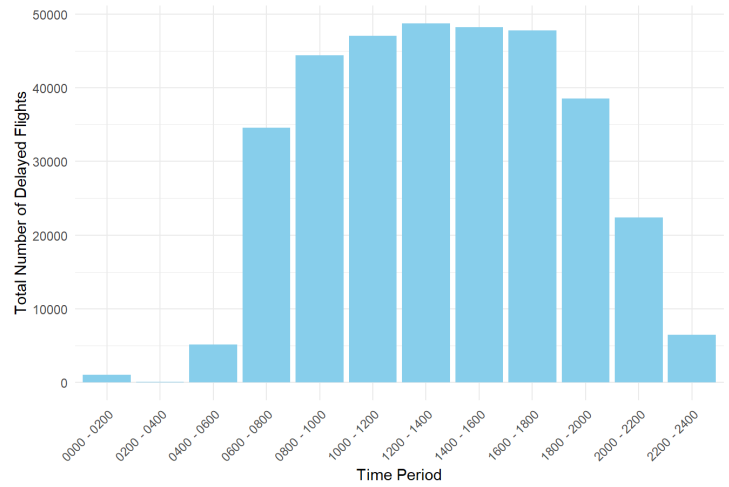
```
## Best time to fly in 2004 : 0200 - 0400
## Best time to fly in 2005 : 0200 - 0400
## Best time to fly in 2006 : 0200 - 0400
## Best time to fly in 2007 : 0200 - 0400
## Best time to fly in 2008 : 0200 - 0400
```

I will then create barplots to visualize dataset for every Saturday of each year.

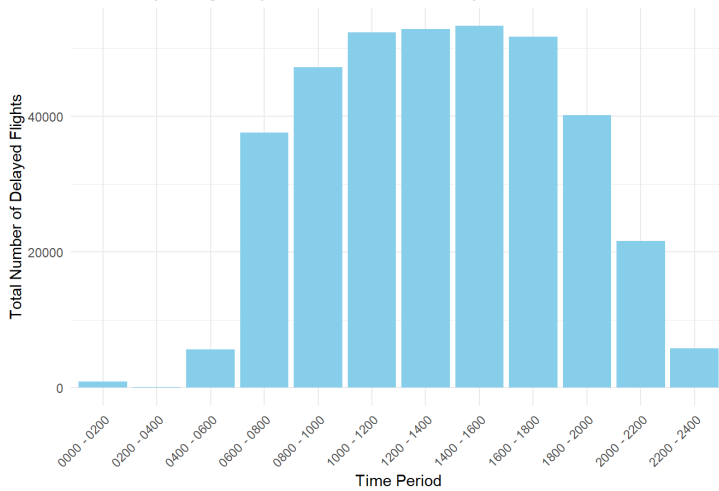
Total Delayed Flights by Time Period for Saturday in Year 2004



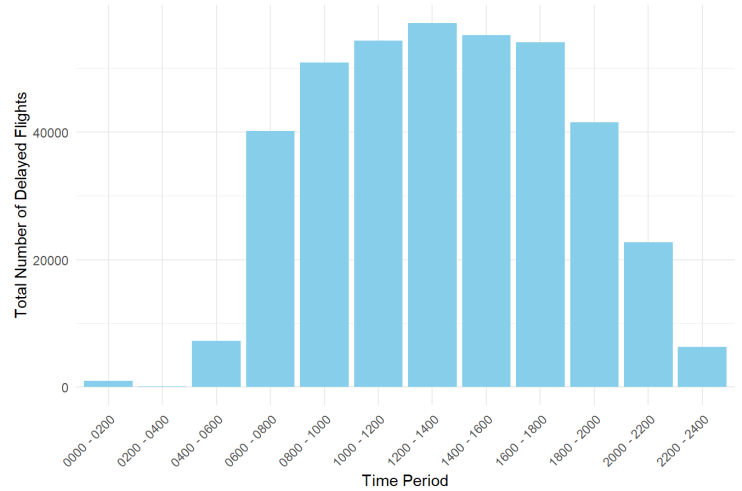
Total Delayed Flights by Time Period for Saturday in Year 2005



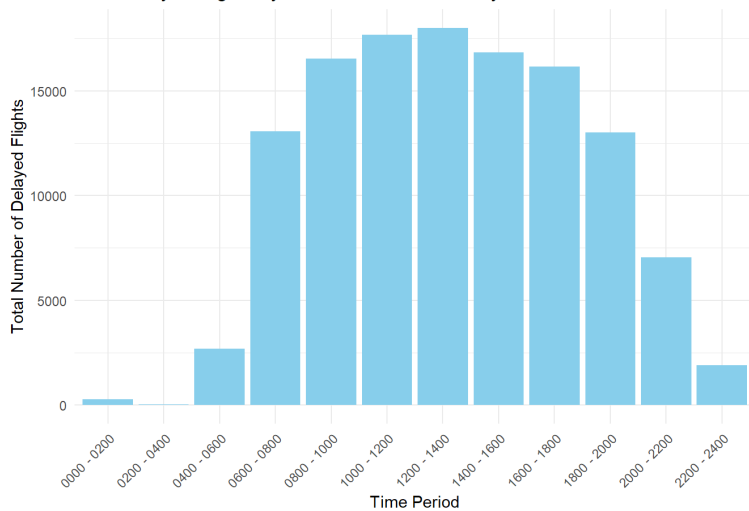
Total Delayed Flights by Time Period for Saturday in Year 2006



Total Delayed Flights by Time Period for Saturday in Year 2007



Total Delayed Flights by Time Period for Saturday in Year 2008



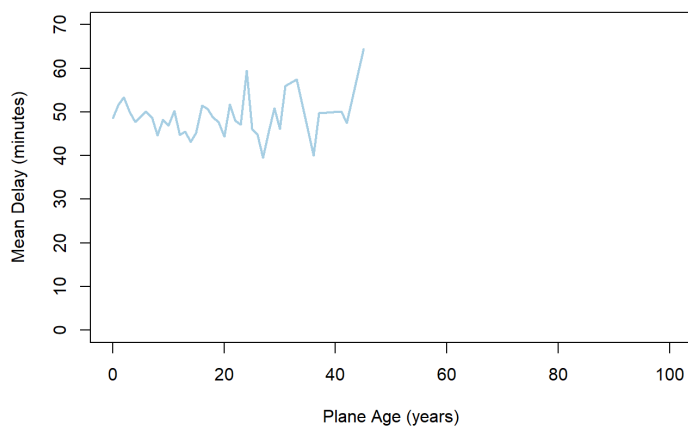
For part b)

For Rmarkdown, I extract the year from IssueYear, and convert the manufacture year to integer. I calculate plane's age as IssueYear minus year. I also filter planes between ages 0-100 years. Next, I query flight data from the database table, and select year, tail number, departure delay and total delay(sum of both).

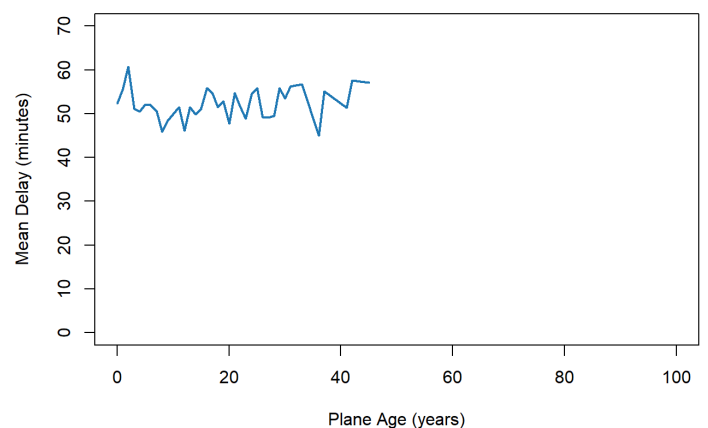
Flights will be filtered with positive delay values and convert delay values to numerical format. I also use RcolorBrewer to set up a color palette. Next, I create plot with appropriate axes. Additionally I filter the delay data for the specific year, merge with plane data using TailNum(tailnumber), calculate mean delays by plane age. For the line plot, we can observe the relationship between plane age and the mean delay.

Here are the line plots for 2004-2008:

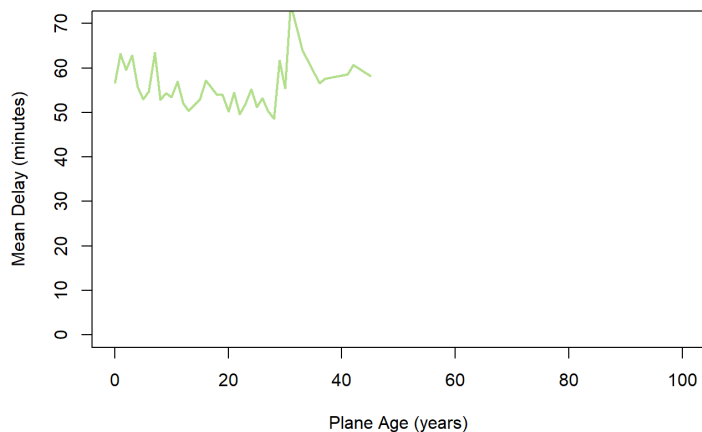
2004 Flight Delays by Plane Age



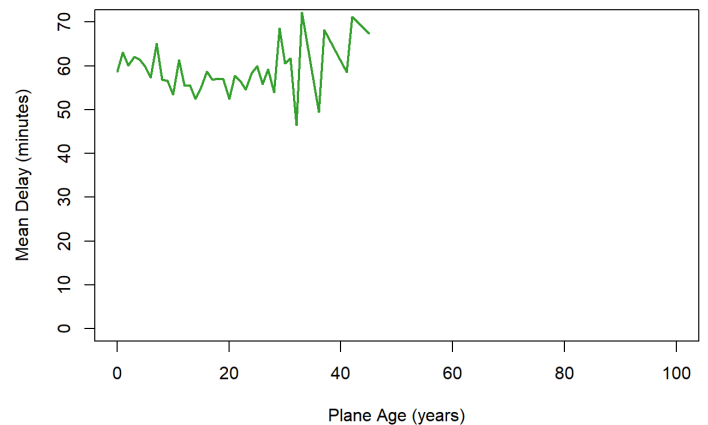
2005 Flight Delays by Plane Age



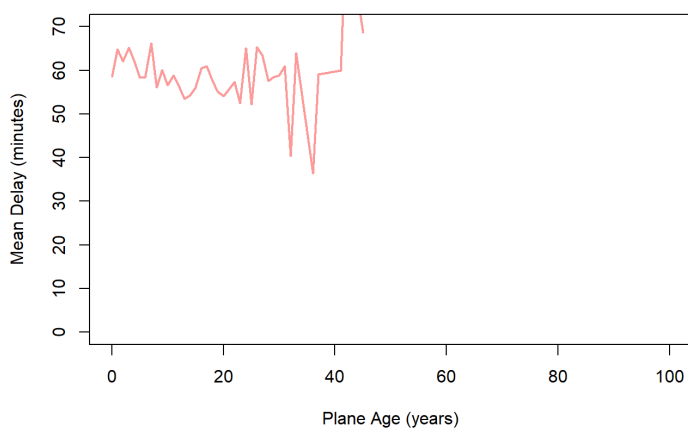
2006 Flight Delays by Plane Age



2007 Flight Delays by Plane Age

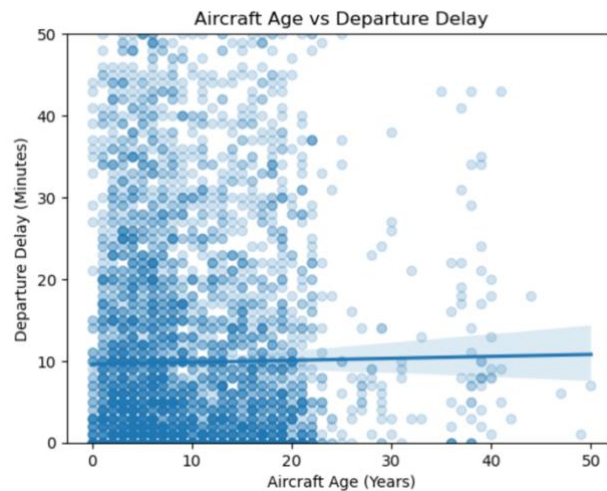


2008 Flight Delays by Plane Age



We can conclude from Rmarkdown analysis that older planes does not mean a higher delay. From 2004-2008, most of the plane regardless of age have a consistent amount of delays in that year. Technology advancements along the years results in newer planes having lesser delays.

For Python, I filtered aircraft aged 0-50 years old. I used a regression plot for overall aircraft age and departure, instead of individual years like in Rmarkdown. I sampled 10,000 flights for 2004-2008.



We can conclude from the python analysis that the mean delay of 2004-2008 flights are higher for older planes.

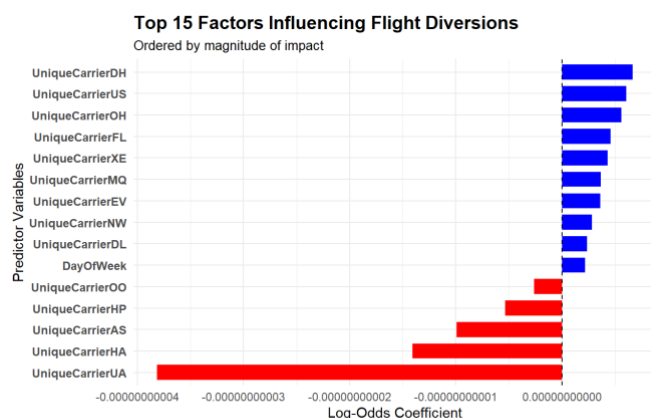
For part c)

For Rmarkdown,

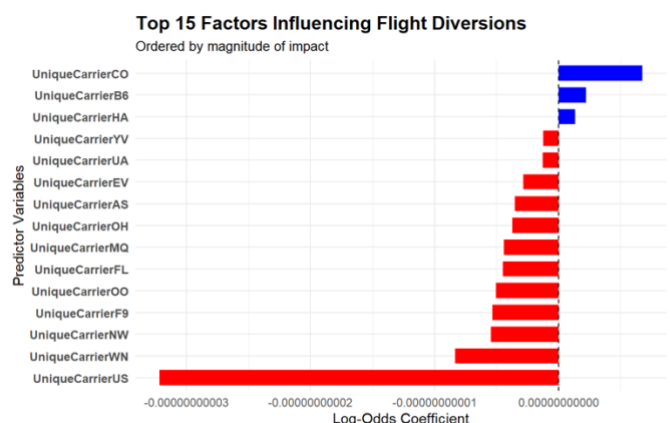
I will be using airport data in conjunction with the respective flight data from 2004-2008. Columns such as departure times and arrival times are converted to numeric values using `numeric_cols`. From the airport data, we can use “iata” which is the airport code from around the world. We are also given longitude and latitude in which both will provide coordinates of the iata. We can use the left join, which is essentially 2 columns shared by 1 column. We share the 2 columns ‘Origin’ and “iata” as both provide geographical coordinates. From this we can derive temporal features from scheduled departure/ arrival times. I have chosen features from the data:

“ new_year, Diverted,Month,DayofWeek,DepTime,CSRDepTime,ArrTime,CSRArrTime,UniqueCarrier,ActualElapsedTime,ArrDelay,DepDelay,Distance,OriginLat,OriginLong,DestLat,DestLong).

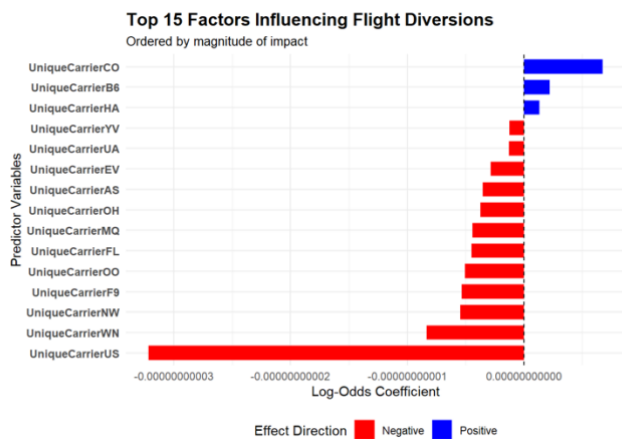
Next, we set up the modeled diversion probability using all available predictors, binomial family with logit link function, and set maximum iterations to ensure convergence. As there are many predictors, the top 15 predictors is narrowed down by coefficient magnitude. I have plotted coefficient against the predictor variables. “Diverted” is the dependent variable whereas the predictor variables are the independent variables. Blue bars represent risk increasing factors and red bars indicate protective factors.



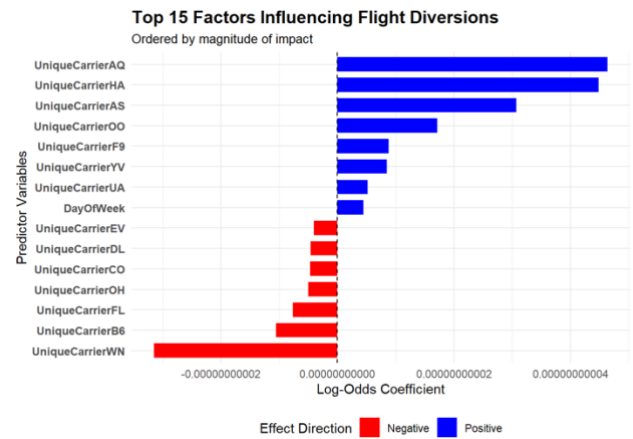
2004



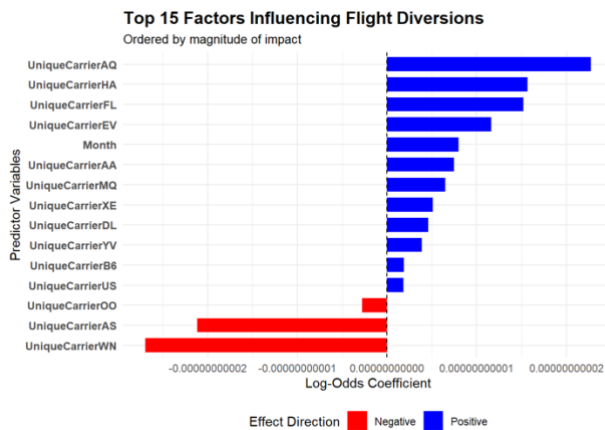
2005



2006



2007



2008

We can conclude from the graph that since the top 15 predictors by coefficient magnitude are all unique carriers, it suggests that carrier operations are the dominating diversion risk predictor out of all other predictor variables.

For python:

Firstly, I convert 'Diverted' to binary form and load the and merge the airport coordinates. Afterwards, I selected features previously stated to be chosen. Using `scaler=StandardScaler()` to scale features selected.

Next, I use a logistic regression model that was trained to identify factors influencing diversions.

I decided to use a barplot to best represent the regression coefficients.

Carrier operations are shown to be the strongest predictors of diversions.